Bắp nước thanh toán và báo cáo

Tài liệu dành cho người phụ trách danh mục đồ ăn, xác nhận tiền và theo dõi đơn tại quầy.

Bạn sẽ đọc được đường đi của một lựa chọn bắp nước từ trang đặt hàng tới SQL Server, cách tiền mặt hoặc chuyển khoản được xác nhận, và cách màn hình nhân viên thống kê đơn. Các tên hàm giữ đúng theo mã nguồn; chữ “báo cáo” trong tài liệu chỉ phần thống kê hiện có.

Quy ước đường dẫn: UI/... nghĩa là src/main/resources/UI/...; tên file Java không ghi thư mục nằm dưới src/main/java/com/cinesnack/ theo controller, service, repository, dto hoặc entity. Các đoạn trích bên dưới luôn ghi đường dẫn đầy đủ và số dòng.

1 Bản đồ luồng

Bắp nước

1. **Giao diện:** src/main/resources/UI/html/booking.html — nạp api.js, cinema.js và booking.js, tạo vùng chọn sản phẩm và combo
2. **Tải danh mục:** src/main/resources/UI/assets/js/booking.js — gọi GET /api/snacks và hienFood() tạo nút tăng giảm
3. **Định tuyến:** CatalogController → CatalogService — layBapNuoc() chuyển yêu cầu xuống lớp đọc dữ liệu
4. **Đọc SQL:** CatalogRepository.layBapNuoc() — chỉ lấy SAN_PHAM và COMBO còn HOAT_DONG
5. **Đưa vào đơn:** cinema.js → checkout.js → CreateOrderRequest → OrderService — gửi id và số lượng; máy chủ đọc lại giá thật rồi tính tổng
6. **Lưu:** OrderRepository.themBapNuoc() — ghi CHI_TIET_HOA_DON_BONG_NUOC với tên và đơn giá tại lúc mua

Thanh toán

1. **Mở đơn:** admin.html → admin.js → orders.js — nhân viên xem chi tiết, tiền cần thu và nút xác nhận
2. **QR:** orders.js → QrPaymentController → QrPaymentService — tạo URL ảnh QR với amount đúng bằng tổng tiền và nội dung là mã đơn
3. **Xác nhận:** orders.js → PaymentController → PaymentService — nhân viên nhập tiền mặt hoặc mã giao dịch và số tiền chuyển khoản thực nhận
4. **Ghi dữ liệu:** PaymentRepository → THANH_TOAN — lưu giao dịch thành công; OrderRepository đổi trạng thái đơn và chi tiết
5. **Phát hành hóa đơn:** PaymentService → InvoiceService → InvoiceRepository — hóa đơn vé/bắp nước chỉ sinh sau xác nhận thanh toán

Thống kê tại trang nhân viên

admin.js gọi GET /api/admin/orders; AdminOrderController → OrderService → OrderRepository truy vấn DON_HANG theo trang, từ khóa và trạng thái. JavaScript đếm các đơn trong mảng vừa nhận để hiển thị ba ô thống kê. Không có API doanh thu tổng hợp, biểu đồ hoặc báo cáo xuất file trong mã nguồn hiện tại.

2 Vai trò từng file

File	Việc file thực hiện và hàm cần đọc
UI/html/booking.html	Vùng hiển thị chọn bắp nước; nạp các script. Không tự truy cập SQL.

UI/assets/js/booking.js	Tải /api/snacks; hienFood() tạo dòng và nút +/−; cập nhật giỏ g.snacks/g.combos.
UI/assets/js/cinema.js	tinhTien() tính tạm trên trình duyệt; gioHopLe() kiểm tra hình dạng giỏ; taoYeuCau() chỉ gửi id và số lượng.
UI/assets/js/checkout.js	Chọn API tạo đơn theo vai trò; không gửi giá tự tính làm giá chính thức.
CatalogController.java	GET /api/snacks nhận yêu cầu công khai.
CatalogService.java	layBapNuoc() chuyển xuống repository; điểm nối controller với dữ liệu.
CatalogRepository.java	SELECT sản phẩm và combo HOAT_DONG; ánh xạ SQL sang Catalog.Food.
BookingCatalogRepository.java	timSanPham()/timCombo() đọc lại giá và trạng thái lúc chốt đơn.
OrderService.java	taoDongBapNuoc() kiểm tra còn bán; tính tổng vé và bắp nước.
OrderRepository.java	themBapNuoc() ghi chi tiết; tinhTongChiTiet() kiểm tra tổng trước thu tiền.
UI/html/admin.html	Màn nhân viên: danh sách đơn, monitor, hóa đơn và ba ô thống kê.
UI/assets/js/admin.js	loadOrders() tìm/lọc/phân trang; monitor cập nhật 10 giây; đếm trạng thái trên trang.
UI/assets/js/orders.js	hienChiTiet() dựng form tiền mặt, chuyển khoản, QR và hóa đơn.
QrPaymentController.java	GET payment-qr cho tài khoản hoặc khách trong phiên.
QrPaymentService.java	Chỉ tạo URL QR; không kết nối ngân hàng để tự nhận callback.
PaymentController.java	POST /cash và /transfer; kiểm tra DTO và chuyển cho service.
PaymentService.java	Khóa đơn, so tiền, kiểm tra thời hạn, lưu thanh toán, đổi trạng thái và gọi phát hành hóa đơn.
PaymentRepository.java	INSERT/SELECT bảng THANH_TOAN; mã giao dịch là dữ liệu đối chiếu.
database/cinesnack.sql	Định nghĩa SAN_PHAM, COMBO, CHI_TIET_HOA_DON_BONG_NUOC, THANH_TOAN và ràng buộc.

3 Đọc các đoạn lệnh quan trọng

Mã nguồn: src/main/resources/UI/assets/js/booking.js · dòng 183–189

```
183    function hienFood() {
184      o("chon-combo").replaceChildren();
185      for (const [key, idKey, ds] of [
186        ["combos", "comboId", food.combos],
187        ["snacks", "productId", food.products],
188      ])
189        for (const h of ds) {
```

Giải thích: Vòng lặp ngoài xử lý hai loại: combos và snacks. Mỗi phần tử h có id, name, price. idKey là tên trường id cần đưa vào giỏ: comboId hoặc productId.

Mã nguồn: src/main/resources/UI/assets/js/booking.js · dòng 208–223

```
208          const thayDoi = (n) => {
209            const old = g[key].find((v) => v[idKey] === h.id);
210            const q = (old?.quantity || 0) + n;
211            if (q < 0 || q > 10) return;
212            g[key] = g[key].filter((v) => v[idKey] !== h.id);
213            if (q)
214              g[key].push({
```

```
215               [idKey]: h.id,
216               name: h.name,
217               quantity: q,
218               unitPrice: h.price,
219             });
220           output.textContent = q;
221           tru.disabled = q === 0;
222           cong.disabled = q === 10;
223           capNhat();
```

Giải thích: find tìm mặt hàng đã chọn; ?. tránh lỗi khi chưa có; filter bỏ dòng cũ, rồi push dòng mới với số lượng q. capNhat() làm mới tổng tạm. Giới hạn 0–10 được chặn ngay tại giao diện.

Mã nguồn: src/main/java/com/cinesnack/repository/CatalogRepository.java · dòng 113–132

```
113     public Catalog.Snacks layBapNuoc() {
114         var sanPham = truyVan.query(
115             "SELECT TOP(200) san_pham_id,ten_san_pham,gia_ban FROM SAN_PHAM WHERE
trang_thai='HOAT_DONG' ORDER BY san_pham_id",
116             (d, i) ->
117                 new Catalog.Food(
118                     d.getLong("san_pham_id"),
119                     d.getString("ten_san_pham"),
120                     d.getBigDecimal("gia_ban")
121                 )
122         );
123         var combo = truyVan.query(
124             "SELECT TOP(200) combo_id,ten_combo,gia_ban FROM COMBO WHERE trang_thai='HOAT_DONG' ORDER
BY combo_id",
125             (d, i) ->
126                 new Catalog.Food(
127                     d.getLong("combo_id"),
128                     d.getString("ten_combo"),
129                     d.getBigDecimal("gia_ban")
130                 )
131         );
132         return new Catalog.Snacks(sanPham, combo);
```

Giải thích: SELECT TOP(200) giới hạn số sản phẩm; WHERE trang_thai='HOAT_DONG' loại mặt hàng ngừng bán. JdbcTemplate biến từng dòng thành Catalog.Food.

Mã nguồn: src/main/java/com/cinesnack/service/OrderService.java · dòng 112–127

```
112                 )
113             );
114         }
115         for (var luaChon : yeuCau.dsCombo()) {
116             dsMatHang.add(
117                 taoDongBapNuoc(
118                     null,
119                     luaChon.maCombo(),
120                     danhMucDatVe.timCombo(luaChon.maCombo()),
121                     luaChon.soLuong()
122                 )
123             );
124         }
125         BigDecimal tongTien = giaVe.multiply(BigDecimal.valueOf(yeuCau.dsMaGhe().size()));
126         for (var dongChiTiet : dsMatHang)
127             tongTien = tongTien.add(
```

Giải thích: Giá chính thức lấy từ matHang.gia() do máy chủ đọc lại. BigDecimal.multiply() nhân giá với số lượng và add() cộng vào tổng; giá trình duyệt không quyết định số tiền thanh toán.

Mã nguồn: src/main/java/com/cinesnack/service/QrPaymentService.java · dòng 61–75

```
 61         try {
 62             tien = don.tongTien().longValueExact();
 63         } catch (ArithmeticException e) {
 64             throw new ResponseStatusException(HttpStatus.CONFLICT, "Số tiền QR phải là VND nguyên");
 65         }
 66         if (tien <= 0) throw new ResponseStatusException(HttpStatus.CONFLICT, "Số tiền QR phải lớn hơn
0");
 67         String noiDung = don.ma();
 68         String url = "https://img.vietqr.io/image/" + maNganHang + "-" + soTaiKhoan +
```

```
            "-qr_only.svg?amount=" + tien + "&addInfo=" + maHoa(noiDung) +
            "&accountName=" + maHoa(tenChuTaiKhoan);
        return new QrPayment(don.tongTien(), noiDung, url);
    }

    private String maHoa(String giaTri) {
        return URLEncoder.encode(giaTri, StandardCharsets.UTF_8);
```

Giải thích: longValueExact() đòi số tiền VND nguyên; URL QR chứa amount và addInfo = mã đơn. maHoa() mã hóa nội dung để đưa an toàn vào URL. Đây là dữ liệu để khách quét, chưa chứng minh ngân hàng đã nhận tiền.

Mã nguồn: src/main/java/com/cinesnack/service/PaymentService.java · dòng 48–65

```
    public Receipt thanhToanTaiQuay(String maDonHang, BigDecimal tienKhachDua, String thuDienTu) {
        if (!dichVuNguoiDung.layTaiKhoanHoatDong(thuDienTu).laNhanVien()) {
            throw new ResponseStatusException(
                HttpStatus.FORBIDDEN,
                "Chỉ nhân viên được xác nhận tiền mặt"
            );
        }
        if (
            tienKhachDua == null ||
            tienKhachDua.signum() < 0 ||
            tienKhachDua.stripTrailingZeros().scale() > 0 ||
            tienKhachDua.compareTo(new BigDecimal("999999999999")) > 0
        ) {
            throw new ResponseStatusException(
                HttpStatus.BAD_REQUEST,
                "Số tiền khách đưa không hợp lệ"
            );
        }
```

Giải thích: @Transactional gộp các bước thu tiền trong một giao dịch. Kiểm tra tài khoản nhân viên, số tiền nguyên, không âm và trong giới hạn trước khi khóa đơn. Admin cũng được coi là nhân viên ở User.laNhanVien() khi xác nhận tiền.

Mã nguồn: src/main/java/com/cinesnack/service/PaymentService.java · dòng 108–127

```
    public Payment xacNhanChuyenKhoan(String maDonHang, String maGiaoDich, BigDecimal soTienNhan,
String thuDienTu) {
        if (!dichVuNguoiDung.layTaiKhoanHoatDong(thuDienTu).laNhanVien()) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Chỉ nhân viên được xác nhận chuyển
khoản");
        }
        if (maGiaoDich == null || !maGiaoDich.matches("[A-Za-z0-9_-]{4,100}")) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Mã giao dịch không hợp lệ");
        }
        if (soTienNhan == null || soTienNhan.signum() <= 0 ||
            soTienNhan.stripTrailingZeros().scale() > 0) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Số tiền đã nhận không hợp lệ");
        }
        Order donHang = khoDonHang.timVaKhoa(maDonHang);
        if (donHang == null) throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Không tìm thấy
đơn hàng");
        if ("DA_THANH_TOAN".equals(donHang.trangThai())) {
            Payment daCo = khoThanhToan.timThanhToanThanhCong(donHang.maSo());
            if (daCo != null && "CHUYEN_KHOAN".equals(daCo.phuongThuc()) &&
                maGiaoDich.equals(daCo.maGiaoDich()) &&
                soTienNhan.compareTo(donHang.tongTien()) == 0 &&
                daCo.soTien().compareTo(donHang.tongTien()) == 0) return daCo;
            throw new ResponseStatusException(HttpStatus.CONFLICT, "Đơn đã được thanh toán");
```

Giải thích: Chuyển khoản phải có mã giao dịch hợp lệ và số tiền đúng bằng tổng đơn. Nếu đơn đã thanh toán, chỉ trả lại giao dịch cũ khi phương thức, mã và số tiền đều trùng; khác thì báo 409.

Mã nguồn: src/main/resources/UI/assets/js/admin.js · dòng 72–79

```
  async function loadOrders() {
    const generation = ++listGeneration;
    const q = o("quan-tri-don-tim").value.trim(),
      status = o("don-trang-thai").value;
    const ds = await CineSnackApi.goi(
```

```
 77
`/api/admin/orders?page=${page}&size=20&q=${encodeURIComponent(q)}&status=${encodeURIComponent(status)}`,
 78      );
 79      if (generation !== listGeneration) return;
```

Giải thích: loadOrders() gọi API với page, size, q và status; encodeURIComponent mã hóa chuỗi tìm kiếm. Kết quả ds là tối đa 20 đơn trên trang, không phải toàn bộ đơn trong hệ thống.

Mã nguồn: src/main/resources/UI/assets/js/admin.js · dòng 109–118

```
109      o("thong-ke").replaceChildren();
110      for (const [label, count] of [
111        ["Đơn trên trang", ds.length],
112        ["Chờ thanh toán", ds.filter((d) => d.status === "CHO_THANH_TOAN").length],
113        ["Đã thanh toán", ds.filter((d) => d.status === "DA_THANH_TOAN").length],
114      ]) {
115        const card = e("article", undefined, "workspace-stat");
116        card.append(e("span", label), e("strong", String(count)));
117        o("thong-ke").append(card);
118      }
```

Giải thích: filter giữ đơn theo trạng thái rồi length đếm. Vì ds chỉ là một trang, ba ô chỉ là thống kê trên trang. Đừng diễn giải thành doanh thu hoặc tổng toàn rạp.

4 Trường hợp cần nhớ

- **Chỉ mua bắp nước:** booking.js có lựa chọn 'Chỉ mua bắp nước'; OrderService cho phép không có ghế nếu có sản phẩm hoặc combo.
- **Sản phẩm hết bán:** OrderService.taoDongBapNuoc() từ chối khi repository trả null hoặc trạng thái khác HOAT_DONG.
- **Tiền mặt:** Có thể đưa nhiều hơn tổng đơn; PaymentService trả lại change để nhân viên trả tiền thừa.
- **Chuyển khoản:** Số tiền thực nhận phải đúng tuyệt đối với tổng đơn; không có cơ chế tiền thừa.
- **QR:** Ảnh QR không tự đổi trạng thái đơn; nhân viên phải đối chiếu giao dịch thực tế.
- **Monitor:** admin.js chỉ tải lại danh sách 10 giây một lần khi đang mở mục monitor và tab còn hiển thị.
- **Báo cáo:** Chưa có báo cáo doanh thu tổng hợp; phần thống kê hiện có chỉ dựa vào 20 đơn được tải của trang hiện tại.

5 Ví dụ số tiền để tự đối chiếu

Giả sử đơn chỉ có hai bắp giá 35.000 đồng mỗi phần: OrderService tính 2 × 35.000 = 70.000 đồng và lưu tổng vào DON_HANG. Nếu khách đưa tiền mặt 100.000 đồng, PaymentService yêu cầu ít nhất 70.000 đồng và trả change = 30.000 đồng; THANH_TOAN ghi số tiền của đơn là 70.000 đồng. Nếu khách chọn chuyển khoản, QR mang amount = 70.000; nhân viên chỉ xác nhận khi thực nhận đúng 70.000 và có mã giao dịch đối chiếu. Không lấy 100.000 làm số tiền chuyển khoản hoặc giá hóa đơn.

Để tự lần theo khi có lỗi: mở Network của trình duyệt, xem URL và mã HTTP; đọc controller tương ứng; theo lời gọi sang service; xem truy vấn repository và bảng SQL. Ví dụ GET /api/snacks trả danh sách rỗng thì kiểm tra SAN_PHAM/COMBO có bản ghi HOAT_DONG trước khi xem code hiển thị.

6 Từ vựng kỹ thuật

- **GET và POST:** GET đọc dữ liệu; POST gửi hành động tạo đơn hoặc xác nhận thanh toán.
- **@RestController:** Spring Boot tự biến giá trị trả về thành JSON cho JavaScript.
- **JdbcTemplate:** Lớp hỗ trợ chạy SQL; dấu ? là tham số, không ghép trực tiếp dữ liệu người dùng vào câu lệnh.
- **BigDecimal:** Kiểu số thập phân phù hợp cho tiền, tránh sai số thường gặp của số thực.
- **@Transactional:** Các bước lưu tiền, đổi trạng thái và tạo hóa đơn cùng thành công hoặc cùng hoàn tác khi lỗi.